ACKNOWLEDGEMENT

Completing this internship and project has been a meaningful journey for me, both academically and personally. First and foremost, I am deeply grateful to my parents. Their constant support, patience, and belief in me have been the strongest pillars throughout this journey. Even during difficult phases, their encouragement kept me grounded and motivated.

I would like to sincerely thank my external guide, **Adarsh Gupta**, for his guidance and support during the development of this project. His practical insights and direction helped me approach problems with more clarity and confidence. I am equally thankful to my internal guide, **Prof. Madhuri Parekh**, whose consistent encouragement, timely feedback, and academic guidance played an important role in shaping this work and keeping me on the right track.

I am also thankful to **FICE Education Pvt. Ltd.** for organizing the **Microsoft Elevate Internship Program.** The exposure to real-world concepts and hands-on learning in areas like Artificial Intelligence and Deep Learning made a significant difference in how I approached this project. It helped me move beyond theory and actually understand how these technologies are applied in practice.

I would also like to acknowledge my friends and peers. Whether it was through discussions, suggestions, or simply being there during stressful times, their presence made this journey smoother and more enjoyable.

Lastly, I would like to thank everyone who, in one way or another, contributed to this experience. Even small interactions and support along the way have had a lasting impact on my learning.

Sincerely,

Jhaveri Vivek Gaurang

ABSTRACT

This report presents the work carried out during the 12-week Microsoft Elevate GTU Internship Program conducted at FICE Education Pvt. Ltd., along with the development of a project titled "Brain Tumor Detection." The internship provided practical exposure to various domains such as Artificial Intelligence, Data Analytics, Machine Learning, Deep Learning, and Microsoft Azure Cloud, which helped in building the foundation required for developing the project. The main objective of the project is to design a system that can classify brain tumors from MRI images using deep learning techniques. In traditional methods, MRI analysis is performed manually by medical professionals, which can be time-consuming and may vary depending on individual experience. This project attempts to address this issue by creating a system that can provide faster and more consistent predictions. For this purpose, a Convolutional Neural Network based on the VGG16 architecture has been used with the help of transfer learning. The pre-trained model is adapted for this specific task by adding custom layers for classification. The dataset used for training consists of more than 7,000 MRI images, which are categorized into four classes: Glioma, Meningioma, Pituitary Tumor, and No Tumor. Before training, preprocessing steps such as resizing and normalization are applied to ensure that all input images are consistent. The model is trained and evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score. The final model achieves an accuracy of approximately 90%, which shows that it is able to classify MRI images with good reliability. Additional evaluation methods such as confusion matrix and ROC curve are also used to understand the performance of the model in more detail. To make the system usable, the trained model is integrated into a web application. The backend is developed

using the Flask framework, which handles image processing and prediction through an API. The frontend is designed using HTML, CSS, and JavaScript, allowing users to upload MRI images and view the prediction results in a simple and clear format. This report includes all stages of the project, starting from problem understanding and data preparation to model development, system design, implementation, and testing. It also highlights some of the challenges faced during the development process, such as handling image variations and improving model performance, along with the steps taken to resolve them.

In conclusion, the project demonstrates how deep learning can be applied to medical image analysis in a practical way. Although the system is not intended to replace medical professionals, it can serve as a supportive tool for faster analysis. The project can be further improved by using larger datasets, adding more features, and deploying the system for real-world use.

LIST OF FIGURES

Figure 4.1	Data Flow Diagram (Level 0 and Level 1)	16
Figure 4.2	Use Case Diagram	18
Figure 4.3	Entity Relationship Diagram	19
Figure 5.1	Frontend – Home Page	22
Figure 5.2	Frontend – Tumor information Page	22
Figure 5.3	Frontend – Upload Page	22
Figure 5.4	Frontend – Tumor Detected Page	23
Figure 5.5	Frontend – No Tumor Detected Page	23
Figure 6.1	Model Training Accuracy and Loss Graph	29
Figure 6.2	Confusion Matrix – Multi-class Classification	30
Figure 6.3	ROC Curve – All Classes	31
Figure 6.4	System Output – Tumour Detected	32
Figure 6.5	System Output – No Tumour Detected	33

LIST OF TABLES

Table 2.1 Major Technical Specifications of Tools Used 05
Table 3.1 Project Scheduling – Gantt Chart Summary 11
Table 4.1 Functional Requirment 13
Table 4.2 Non – Functional Requirment 14
Table 5.1 Dataset Distribution by Class 20
Table 5.2 Model Architecture Details 26
Table 7.1 Model Performance Metrics by Class 34

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
AZ-900	Microsoft Azure Fundamentals Certification
CNN	Convolutional Neural Network
DAX	Data Analysis Expressions
DL	Deep Learning
EDA	Exploratory Data Analysis
FICE	Foundation for Innovation and Creativity in Education
GTU	Gujarat Technological University
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IaaS	Infrastructure as a Service
IoU	Intersection over Union
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
KPI	Key Performance Indicator
ML	Machine Learning
MRI	Magnetic Resonance Imaging
MS	Microsoft
PaaS	Platform as a Service
PCA	Principal Component Analysis
REST	Representational State Transfer
ReLU	Rectified Linear Unit
RU	Request Unit (Azure Cosmos DB)
SaaS	Software as a Service
SAS	Shared Access Signature
SVM	Support Vector Machine
VGG16	Visual Geometry Group 16-layer network

TABLE OF CONTENTS

Acknowledgement i
Abstract ii
List of Figures iii
List of Tables iv
List of Abbreviations v
Table of Contents vi

Chapter 1 Overview of the Organization 1

1.1 About FICE Education Pvt. Ltd. 1
1.2 Microsoft Elevate GTU Internship Program 1
1.3 Scope of Work 1
1.4 Organization Structure and Mentorship 2

Chapter 2 Overview of Learning Modules 2

2.1 Data Analytics and Python Foundation (Weeks 1–2) 2
2.2 Power BI and DAX (Weeks 3–4) 2
2.3 Machine Learning (Weeks 5–6) 4
2.4 Deep Learning (Weeks 7–8) 4
2.5 Microsoft Azure Cloud (Weeks 9–10) 4
2.6 Major Technical Specifications of Tools Used 5

Chapter 3 Introduction to Project 6

3.1 Project Summary 6
3.2 Purpose 6
3.3 Objectives 6
3.4 Scope 7
3.5 System Architecture Overview 7
3.5.1 Presentation Layer 8
3.5.2 Application Layer 8
3.5.3 Data Layer 8
3.6 Project Planning and Management 9
3.6.1 Development Methodology 9
3.6.2 Effort and Time Estimation 9
3.6.3 Roles and Responsibilities 9
3.7 Technology and Literature Review 10

3.7.1 Evolution of Medical Image Analysis 10
3.7.2 Deep Learning in Healthcare 10
3.7.3 CNN Models for Brain Tumour Detection 10
3.8 Project Scheduling (Gantt Chart Summary) 11

Chapter 4 System Analysis 12

4.1 Study of Current System 12
4.2 Problems and Weaknesses 12
4.3 Proposed System 12
4.4 Requirements of New System 13
4.4.1 Functional Requirements 13
4.4.2 Non-Functional Requirements 14
4.5 System Feasibility 15
4.5.1 Technical Feasibility 15
4.5.2 Economic Feasibility 15
4.5.3 Operational Feasibility 15
4.6 Data Flow Diagrams 15
4.7 System Modules 17
4.8 Use Case Diagrams 17
4.9 Entity Relationship Diagram 18
4.10 Data Models 19

Chapter 5 System Design 20

5.1 Methodology 20
5.2 Dataset Design 20
5.3 System Design 20
5.3.1 Overall System Workflow 21
5.3.2 System Architecture 21
5.4 Frontend Implementation 21
5.5 Backend Implementation 24
5.6 Image Preprocessing 24
5.7 Model Design 24
5.8 Model Architecture Details 25
5.9 Model Training 27
5.10 Model Evaluation 27
5.11 Integration of System 27
5.12 Algorithm Flow 28

Chapter 6 Results and Discussion **29**

6.1 Introduction 29

6.2 Model Training Results 29

6.3 Model Evaluation 29

6.3.1 Confusion Matrix 30

6.3.2 ROC Curve 31

6.4 System Output Results 32

6.5 Performance Analysis 33

6.6 Discussion 33

Chapter 7 Testing **34**

7.1 Testing Plan / Strategy 34

7.2 Test Results and Analysis 34

Chapter 8 Conclusion and Discussion **35**

8.1 Overall Analysis of Internship / Project Viabilities 35

8.2 Photographs and Date of Surprise Visit by Institute Mentor 35

8.3 Problems Encountered and Possible Solutions 35

8.4 Summary of Internship / Project Work 36

8.5 Limitation and Future Enhancement 36

Reference and Bibliography **37**

CHAPTER 1 OVERVIEW OF THE ORGANIZATION

1.1 ABOUT FICE EDUCATION PVT. LTD.

FICE Education Pvt. Ltd. is an educational technology company headquartered at A-11-1105, Unitech Arcadia, South City-2, Gurgaon-122018. The organization specializes in designing and delivering industry-aligned technology programs, skill development initiatives, and internship programs for engineering students across India. In collaboration with Gujarat Technological University (GTU), FICE Education organized the Microsoft Elevate GTU Internship Program, which forms the basis of this internship.

1.2 MICROSOFT ELEVATE GTU INTERNSHIP PROGRAM

The Microsoft Elevate Program is a structured 12-week virtual internship program conducted by FICE Education in partnership with the FICE Foundation and Gujarat Technological University. The program is designed to bridge the gap between academic knowledge and industry readiness by providing engineering students with hands-on exposure to Data Analytics, Machine Learning, Deep Learning, and Microsoft Azure Cloud services. Microsoft-certified trainers delivered live sessions, hands-on labs, and assignment-based evaluations throughout the program.

1.3 SCOPE OF WORK

The program encompassed the following major learning domains delivered over 12 weeks:

- Weeks 1–2: Introduction to Artificial Intelligence, Data Analytics, and Python-based Exploratory Data Analysis
- Weeks 3–4: Power BI, DAX (Data Analysis Expressions), and Dashboard Development
- Weeks 5–6: Machine Learning — Supervised Learning, Unsupervised Learning, and Reinforcement Learning
- Weeks 7–8: Deep Learning — Artificial Neural Networks, Convolutional Neural Networks, TensorFlow/Keras, and Generative AI
- Weeks 9–10: Microsoft Azure Cloud — Blob Storage, Azure Functions, Cosmos DB, DevOps, CI/CD, and Security
- Weeks 11–12: Capstone Project Implementation, Deployment, and Final Presentation

In parallel with the Microsoft Elevate curriculum, the candidate undertook a substantive GTU capstone project titled NeuraScan: Brain Tumour MRI Classification Using Deep Learning, which forms a central deliverable of this report.

1.4 ORGANIZATION STRUCTURE AND MENTORSHIP

The internship was structured with dual mentorship. The external industry mentor, Adarsh, provided guidance on project direction, technical implementation, and program evaluation from FICE Education Pvt. Ltd.. The internal faculty guide, Prof. Madhuri Parekh, SAL Institute of Technology and Engineering Research, provided academic oversight and ensured compliance with GTU internship requirements.

CHAPTER 2 OVERVIEW OF LEARNING MODULES

2.1 DATA ANALYTICS & PYTHON FOUNDATION (WEEKS 1–2)

The first two weeks of the program established a strong foundation in data analysis and Python programming. The sessions covered the full data analytics workflow: data collection, cleaning, exploratory data analysis (EDA), and visualization. Key tools and libraries studied included pandas, numpy, matplotlib, and seaborn. Practical exercises involved handling missing data using imputation techniques, detecting outliers via IQR and Z-score methods, normalization using Min-Max scaling and standardization, and encoding categorical variables.

Visualization skills were developed through line plots, bar charts, scatter plots, heatmaps, and box plots. The candidate gained proficiency in Python-based data wrangling and statistical summaries, which subsequently formed the backbone of the data preprocessing pipeline in the NeuraScan project.

2.2 POWER BI AND DAX (WEEKS 3–4)

Weeks 3 and 4 were dedicated to Microsoft Power BI for business intelligence and data visualization. Sessions covered DAX (Data Analysis Expressions) for creating calculated columns and measures, Power BI visualizations including bar, column, line, pie, scatter, and map charts, and interactive features such as slicers, filters, drill-through, and bookmarks. Advanced DAX topics included iterator functions (SUMX, AVERAGEX), time intelligence functions (DATEADD, SAMEPERIODLASTYEAR), and context manipulation using CALCULATE and ALL.

The candidate completed comprehensive dashboards with KPI cards, Year-over-Year comparisons, and regional performance analytics. These skills contributed to the candidate's ability to design analytical interfaces and interpret model performance metrics in the NeuraScan project.

2.3 MACHINE LEARNING (WEEKS 5–6)

Weeks 5 and 6 provided a thorough treatment of machine learning algorithms using Python's scikit-learn library. Supervised learning topics included Linear Regression, Logistic Regression, Decision Trees, Random Forest, Support Vector Machines (SVM), and K-Nearest Neighbors (KNN). Model evaluation methods covered accuracy, precision, recall, F1-score, confusion matrices, ROC curves, train-test splitting, and k-fold cross-validation.

Unsupervised learning covered K-Means clustering, hierarchical clustering, DBSCAN, and Principal Component Analysis (PCA). Reinforcement Learning fundamentals including Markov Decision Processes, Q-Learning, and policy optimization were also introduced. The candidate completed assignments applying these methods to standard datasets and recognized their relevance to the classification objectives of the NeuraScan project.

2.4 DEEP LEARNING (WEEKS 7–8)

Weeks 7 and 8 formed the most technically intensive phase of the curriculum. Fundamentals of artificial neural networks were covered, including perceptrons, forward propagation, backpropagation, activation functions (ReLU, Sigmoid, Tanh, Softmax), loss functions, and optimizers (SGD, Adam). Hands-on implementation using TensorFlow 2.x and Keras was central to this phase, with the candidate building feedforward networks for MNIST digit classification.

Convolutional Neural Networks (CNNs) were studied in detail, covering convolution operations, pooling layers, feature maps, parameter sharing, and popular architectures including LeNet, AlexNet, and VGG. Transfer learning was studied and applied using VGG16 pre-trained on ImageNet, which directly informed the model architecture chosen for the NeuraScan project. Generative AI concepts — GANs, VAEs, and large language models — were also introduced during this phase.

2.5 MICROSOFT AZURE CLOUD (WEEKS 9–10)

Weeks 9 and 10 provided comprehensive training in Microsoft Azure cloud services. Azure Blob Storage operations — including container creation, access tiers, Shared Access Signatures, lifecycle policies, and the Python SDK — were covered through hands-on labs.

Azure Functions (serverless computing) were studied for event-driven processing, with the candidate building HTTP-triggered, Timer-triggered, and Blob-triggered functions.

Azure Cosmos DB was studied for NoSQL document storage, including SQL API queries, partition key design, Request Units, and Python SDK CRUD operations. Azure DevOps covered CI/CD pipeline design, YAML pipelines, Azure Repos, and Azure Boards. Azure Monitor, Application Insights, and Azure Security (Key Vault, RBAC, Managed Identities) completed the cloud curriculum. The candidate earned the Microsoft Azure Fundamentals (AZ-900) certification with a score of 820/1000 during Week 10.

2.6 MAJOR TECHNICAL SPECIFICATIONS OF TOOLS USED

Tool / Technology	Version / Details	Purpose
Python	3.11	Core development language
TensorFlow / Keras	2.15	Deep learning framework
VGG16	ImageNet pre-trained	Transfer learning backbone
Flask	3.x	REST API backend
scikit-learn	1.x	ML algorithms and evaluation
Power BI Desktop	Latest	Business intelligence dashboards
Microsoft Azure	Cloud platform	Blob Storage, Functions, Cosmos DB
VS Code	Latest	Integrated development environment
GitHub	Version control	Source code management

CHAPTER 3 INTRODUCTION TO PROJECT

3.1 Project Summary

The project titled "NeuraScan: Brain Tumor Detection System" is a deep learning-based web application developed to classify brain tumors using MRI images. The system utilizes a Convolutional Neural Network (CNN) model with transfer learning based on the VGG16 architecture to detect and classify different types of brain tumors.

The application allows users to upload MRI scan images through a web interface. The uploaded image is processed by the backend system, which performs preprocessing and passes it through the trained model. The model then predicts the type of tumor along with confidence scores for each class. The results are displayed to the user through a dedicated result page.

This system demonstrates the practical application of artificial intelligence in the medical domain and aims to assist in early detection and diagnosis.

3.2 Purpose

The primary purpose of this project is to develop an intelligent and automated system capable of detecting brain tumors from MRI images with high accuracy.

The system aims to reduce the dependency on manual analysis, which can be time-consuming and subject to human error. By providing fast and reliable predictions, the system can assist healthcare professionals in making better decisions.

Additionally, the project serves as a learning platform to apply deep learning concepts in real-world applications, particularly in the field of medical image analysis.

3.3 Objectives

The main objectives of the project are as follows:

- To develop a deep learning model capable of classifying brain MRI images into multiple categories.

- To implement transfer learning using the VGG16 architecture for improved performance.
- To design and develop a user-friendly web interface for uploading MRI images and displaying results.
- To build a Flask-based backend API for handling image processing and prediction.
- To integrate frontend and backend components for real-time interaction.
- To achieve high accuracy and reliability in tumor classification.

3.4 Scope

The scope of this project includes the classification of brain MRI images into four categories: Glioma, Meningioma, Pituitary Tumor, and No Tumor.

The system provides real-time predictions through a web-based interface, making it accessible and easy to use. It is designed as a proof-of-concept system demonstrating how artificial intelligence can assist in medical diagnosis.

However, the system has certain limitations. It works only with MRI images and does not support other imaging modalities. The predictions generated by the system are not a substitute for professional medical diagnosis and should be used only for educational or assistive purposes. The accuracy of the model depends on the quality and diversity of the dataset used for training.

3.5 System Architecture Overview

The NeuraScan system is designed using a three-layer architecture consisting of the presentation layer, application layer, and data layer. This modular architecture ensures better scalability, maintainability, and separation of concerns.

Each layer is responsible for a specific set of functionalities, allowing the system to operate efficiently and enabling independent development and modification of components.

3.5.1 Presentation Layer

The presentation layer is responsible for user interaction and is implemented using HTML, CSS, and JavaScript.

It consists of three main web pages: the home page, the upload page, and the result page. The home page provides information about brain tumors and the system. The upload page allows users to select or drag and drop MRI images. The result page displays the prediction along with confidence scores and relevant medical information.

The frontend communicates with the backend using HTTP requests, specifically through the Fetch API, enabling real-time interaction between the user and the system.

3.5.2 Application Layer

The application layer is implemented using the Flask framework in Python and acts as the core processing unit of the system.

This layer handles incoming requests from the frontend, processes the uploaded images, and performs predictions using the trained deep learning model. It includes functionalities such as file validation, image preprocessing, model inference, and response generation.

The primary API endpoint used in this system is the "/predict" endpoint, which accepts image input and returns prediction results in JSON format.

3.5.3 Data Layer

The data layer consists of the trained deep learning model used for classification.

The model is based on the VGG16 architecture and is trained on a dataset of brain MRI images. It accepts input images of size 224×224 pixels and outputs a probability distribution across four classes: Glioma, Meningioma, Pituitary Tumor, and No Tumor.

This layer is responsible for feature extraction and classification, making it the most critical component of the system.

3.6 Project Planning and Management

Project planning and management play a crucial role in ensuring the successful completion of the project within the given time frame. The project was divided into multiple phases, each focusing on a specific aspect such as learning, development, testing, and integration.

A structured approach was followed to manage time effectively and ensure that all objectives were achieved systematically.

3.6.1 Development Methodology

The project follows an iterative development methodology, where the system is developed in multiple stages with continuous improvements.

The development process includes the following phases: learning and understanding concepts, data preparation, model development, backend implementation, frontend development, and testing.

This approach allows for gradual improvement and debugging at each stage, ensuring a stable and efficient final system.

3.6.2 Effort and Time Estimation

The total duration of the project was approximately 12 weeks. The time was distributed across different phases of development.

The initial weeks were dedicated to learning and understanding core concepts. The middle phase focused on model development and training. The later stages involved backend and frontend development, followed by testing and documentation.

This structured allocation of time helped in completing the project efficiently within the given timeline.

3.6.3 Roles and Responsibilities

The project was developed individually by Vivek Jhaveri, who was responsible for all aspects of the project.

The responsibilities included dataset preparation, model development, backend API implementation, frontend design, system integration, testing, and documentation.

Guidance was provided by mentors during the internship program, ensuring that the project met academic and technical standards.

3.7 Technology and Literature Review

The project is based on modern advancements in artificial intelligence and medical imaging. A review of existing technologies and research was conducted to understand the current state of the field.

The use of deep learning, particularly convolutional neural networks, has significantly improved the accuracy of medical image analysis systems.

3.7.1 Evolution of Medical Image Analysis

Medical image analysis has evolved significantly over time. Earlier methods relied on manual observation and traditional image processing techniques, which were limited in accuracy and efficiency.

With the introduction of machine learning and deep learning, automated systems have been developed that can analyze medical images with high precision. These systems reduce human effort and improve diagnostic accuracy.

3.7.2 Deep Learning in Healthcare

Deep learning has become an essential tool in healthcare for tasks such as disease detection, image classification, and pattern recognition.

Convolutional neural networks are particularly effective in analyzing medical images, as they can automatically learn spatial features and patterns from data. This makes them highly suitable for applications such as tumor detection.

3.7.3 CNN Models for Brain Tumor Detection

CNN-based models such as VGG16 are widely used for brain tumor detection due to their ability to extract complex features from images.

These models eliminate the need for manual feature extraction and provide high accuracy in classification tasks. Transfer learning further enhances performance by leveraging pre-trained models.

In this project, VGG16 is used as the base model and fine-tuned for brain tumor classification, resulting in efficient and accurate predictions.

3.8 PROJECT SCHEDULING (GANTT CHART SUMMARY)

Phase	Activity	Duration
Phase 1	MS Elevate Curriculum: AI, Data Analytics, Power BI	Weeks 1–4
Phase 1	MS Elevate Curriculum: Machine Learning, Deep Learning	Weeks 5–8
Phase 2	NeuraScan: Dataset acquisition and preprocessing	Week 8
Phase 2	NeuraScan: VGG16 model training and evaluation	Weeks 9–10
Phase 2	Flask API development and integration	Week 10
Phase 3	Frontend web application development	Week 11
Phase 3	MS Elevate Curriculum: Azure Cloud, DevOps	Weeks 9–10
Phase 3	End-to-end testing, documentation, presentation	Weeks 11–12

CHAPTER 4 SYSTEM ANALYSIS

4.1 Study of Current System

The current system for brain tumor detection is primarily based on manual interpretation of MRI scans by radiologists and medical experts. In this process, specialists examine brain images to identify abnormalities such as tumor presence, size, and type. While this approach benefits from expert knowledge, it is highly dependent on the experience and judgment of the individual performing the diagnosis.

The process is time-consuming, especially when dealing with a large number of patients. In many cases, multiple specialists are required to validate the diagnosis, which increases both time and cost. Furthermore, the availability of skilled professionals is limited in many regions, making timely diagnosis difficult.

Although some advanced hospitals use computer-aided diagnostic systems, these solutions are often expensive and require specialized infrastructure. As a result, they are not widely accessible, particularly in smaller healthcare centers.

4.2 Problems and Weaknesses

The existing system faces several challenges that limit its efficiency and scalability. The manual nature of diagnosis makes the process slow and heavily dependent on human effort. There is also a possibility of errors due to fatigue, oversight, or subjective interpretation of MRI scans.

Another major issue is the high cost of advanced diagnostic tools, which restricts their use to well-equipped hospitals. Additionally, manual analysis becomes increasingly difficult as the volume of medical data grows. These limitations highlight the need for an automated system that can assist in faster and more accurate diagnosis.

4.3 Proposed System

The proposed system is an AI-based brain tumor detection system designed to automate the process of MRI image classification. The system uses a deep learning model based on the VGG16 architecture to analyze MRI scans and classify them into different tumor categories.

The system allows users to upload MRI images through a web interface. The uploaded image is processed using preprocessing techniques and passed to the trained model for prediction. The model generates classification results along with confidence scores, which are displayed to the user.

The proposed system aims to reduce manual effort, improve accuracy, and provide faster results compared to traditional methods.

4.4 Requirements of New System

The proposed system is designed to address the limitations of the current approach by introducing automation and intelligence into the diagnosis process. It aims to provide accurate predictions with minimal human intervention while maintaining ease of use.

4.4.1 Functional Requirements

The system should allow users to upload MRI images through a web interface and process them efficiently. Once the image is uploaded, the system must preprocess it and pass it through a trained deep learning model for classification.

The system should be capable of classifying MRI images into four categories: Glioma, Meningioma, Pituitary Tumor, and No Tumor. After classification, the results should be displayed clearly along with confidence scores.

The application should also handle invalid inputs, such as unsupported file formats or empty uploads, and provide appropriate feedback to the user.

Sr. No	Requirement	Description
1	Image Upload	User uploads MRI image
2	Preprocessing	Resize and normalize image
3	Prediction	Classify tumor using model

Sr. No	Requirement	Description
4	Result Display	Show class + confidence
5	Error Handling	Handle invalid inputs

4.4.2 Non-Functional Requirements

The system should provide quick responses to ensure real-time prediction capability. It must be reliable and produce consistent results across different inputs. The application should be scalable so that it can handle multiple users simultaneously without performance degradation.

Security and privacy are also important considerations, especially when dealing with medical data. The system should be easy to use and accessible through standard web browsers without requiring technical expertise.

Type	Description
Performance	Fast prediction response
Reliability	Consistent output
Scalability	Handle multiple users
Security	Data safety
Usability	Easy interface

4.5 System Feasibility

Feasibility analysis helps determine whether the proposed system is practical and viable for implementation.

4.5.1 Technical Feasibility

The system is technically feasible because it is built using widely available technologies such as Python, TensorFlow, and Flask. These technologies provide strong support for developing machine learning and web-based applications.

The model can be trained using platforms like Google Colab, which provide GPU support without requiring expensive hardware. The frontend is developed using standard web technologies, making it easy to deploy and maintain.

4.5.2 Economic Feasibility

The system is economically feasible as it relies entirely on open-source tools and libraries. There is no need for costly software licenses or specialized hardware.

The development and deployment costs are minimal, making the project suitable for academic use and scalable for future enhancements.

4.5.3 Operational Feasibility

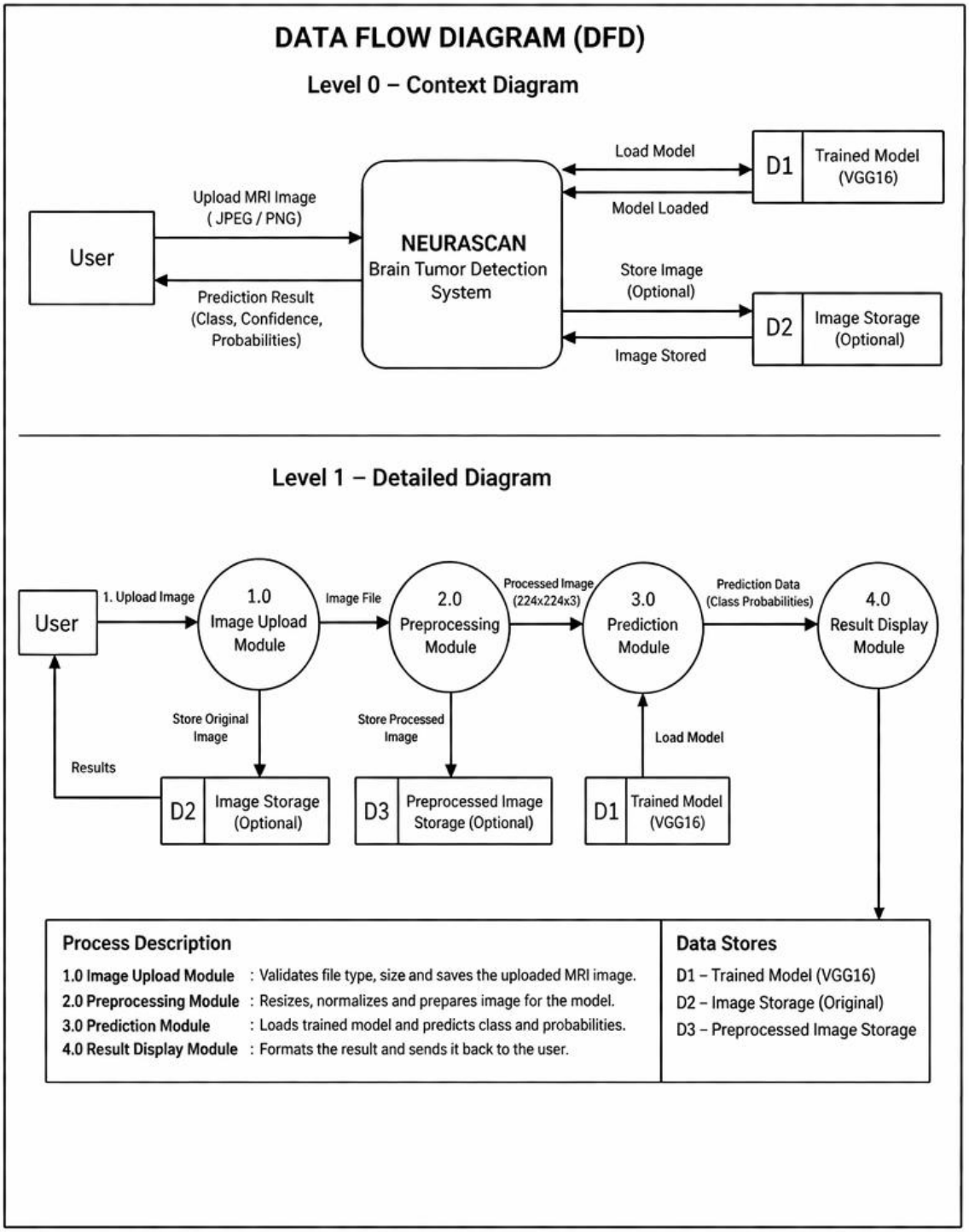
The system is designed to be user-friendly and does not require advanced technical knowledge to operate. Users can easily upload MRI images and obtain results without any training.

The system can be integrated into existing workflows, making it practical for real-world usage. Its simplicity and efficiency make it suitable for both educational and assistive purposes.

4.6 Data Flow Diagrams

The Data Flow Diagram (DFD) illustrates how data moves through the system. It helps in understanding the interaction between different components of the application.

At a high level, the system receives an MRI image from the user, processes it, and returns the prediction result. At a more detailed level, the process involves multiple stages including image upload, preprocessing, model prediction, and result generation.



4.7 System Modules

The system is divided into multiple modules to ensure proper functionality and maintainability.

The image upload module allows users to upload MRI images through the web interface. The preprocessing module processes the image by resizing and normalizing it before passing it to the model.

The prediction module uses the trained deep learning model to classify the image into one of the predefined categories. The result module displays the predicted class along with confidence scores.

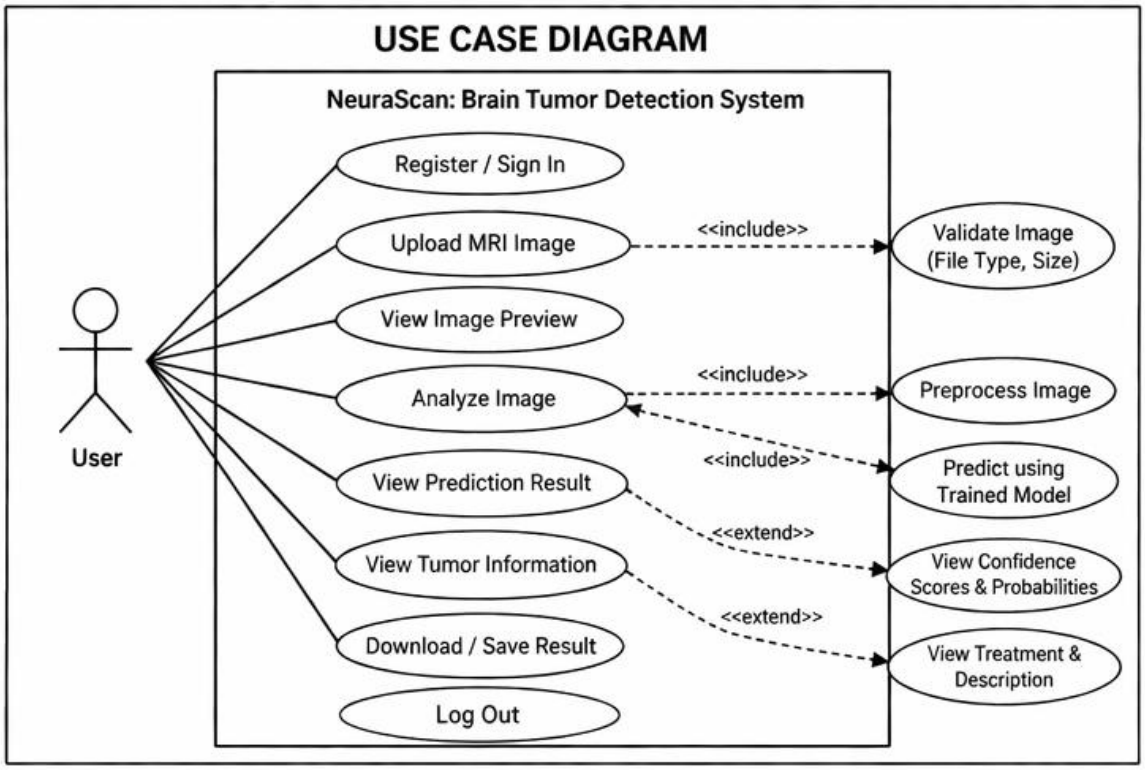
This modular design improves system organization and simplifies debugging and maintenance.

4.8 Use Case Diagrams

The Use Case Diagram represents the interaction between the user and the system. It shows the different functionalities that the system provides to the user.

In this project, the primary actor is the user. The user interacts with the system by uploading an MRI image, initiating the analysis process, and viewing the results.

The diagram should include use cases such as uploading the image, analyzing the image, and viewing the prediction results. These use cases are connected to the user, indicating their interaction with the system.

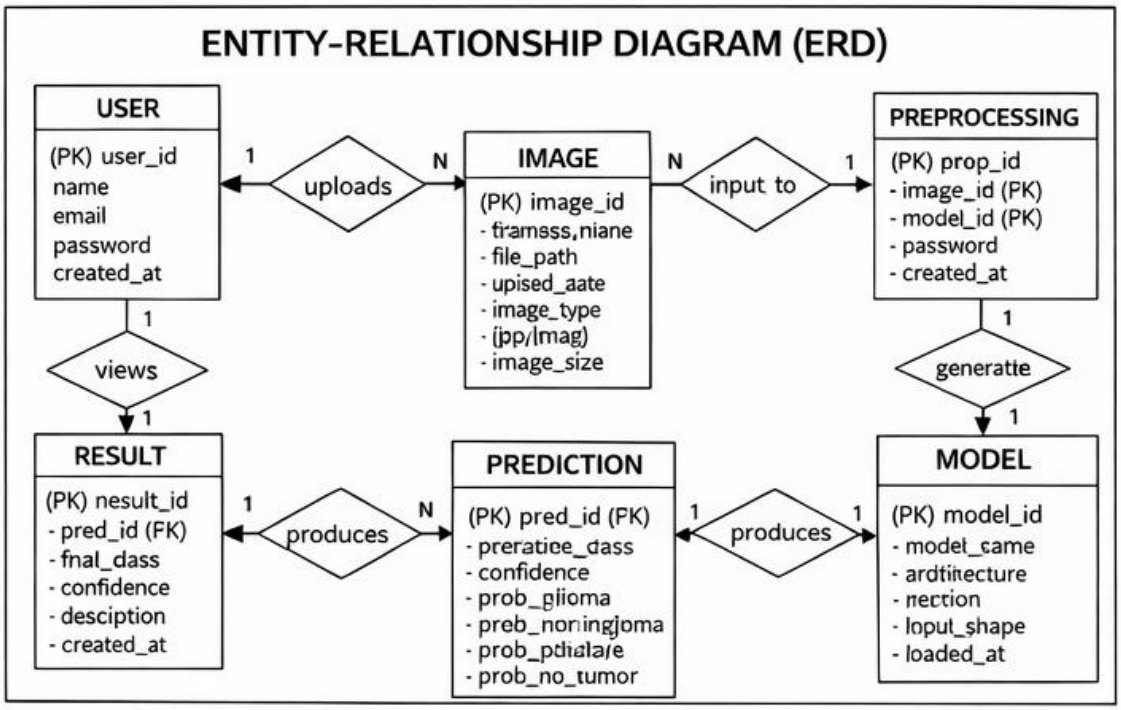


4.9 Entity Relationship Diagram

The Entity Relationship (ER) Diagram represents the logical structure of data used in the system.

Although this project does not use a complex database, a simple ER diagram can be created to represent the relationship between different entities. The main entities include the user, the uploaded image, and the prediction result.

The user uploads an image, and the system generates a prediction based on that image. This relationship can be represented clearly using an ER diagram.



4.10 Data Models

The system uses image data as input and generates prediction data as output. The input data consists of MRI images in formats such as JPEG or PNG, which are resized to a standard dimension of 224×224 pixels before processing.

The output data includes the predicted class of the tumor, the confidence score, and the probability distribution for each class. This structured output allows users to understand the prediction in a clear and informative manner.

CHAPTER 5 SYSTEM DESIGN

5.1 METHODOLOGY

The NeuraScan system is designed as a three-tier web application: a frontend presentation layer, a Flask API middle tier, and a deep learning model inference layer. The architecture decouples the user interface from the model inference logic, enabling independent updates to either component without disrupting the overall system.

5.2 DATASET DESIGN

The training dataset was sourced from the Kaggle Brain MRI Images for Brain Tumor Detection dataset. The dataset contains 7,023 MRI images distributed across four classes: Glioma (1,621 images), Meningioma (1,645 images), Pituitary Tumour (1,757 images), and No Tumour (2,000 images). Images were split into training and validation sets using an 80/20 ratio. Keras ImageDataGenerator was applied for on-the-fly data augmentation including rotation (up to 20 degrees), horizontal flip, zoom (up to 20%), and width/height shifts.

Class	Total Images	Training	Validation
Glioma	1,621	1,297	324
Meningioma	1,645	1,316	329
Pituitary Tumour	1,757	1,406	351
No Tumour	2,000	1,600	400
Total	7,023	5,619	1,404

Table 5.1 Dataset Distribution by Class

5.3 System Design

The system is designed using a layered architecture consisting of frontend, backend, and model layers. Each layer interacts with the other to complete the process of tumor detection.

The frontend handles user interaction, the backend processes requests and communicates with the model, and the model performs classification.

5.3.1 Overall System Workflow

The workflow of the system begins when the user uploads an MRI image through the web interface. The image is then sent to the backend server, where preprocessing is performed. The processed image is passed to the trained deep learning model, which generates predictions. The results are then sent back to the frontend and displayed to the user.

5.3.2 System Architecture

The system consists of three main components:

- The frontend, which provides the user interface and handles user input.
- The backend, which processes requests and performs prediction using the trained model.
- The deep learning model, which classifies the MRI images into different tumor categories.

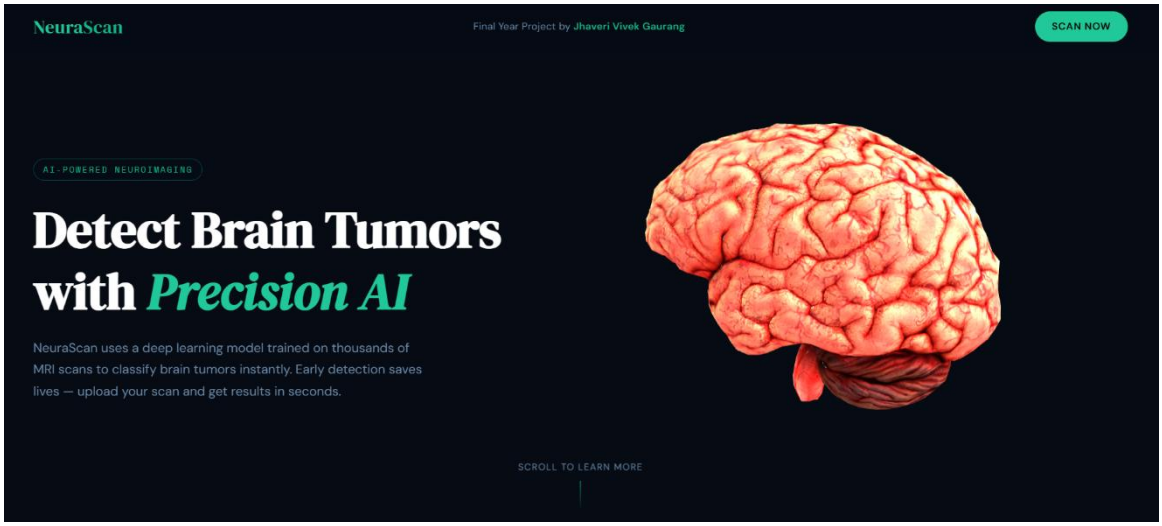
This architecture ensures separation of concerns and improves maintainability.

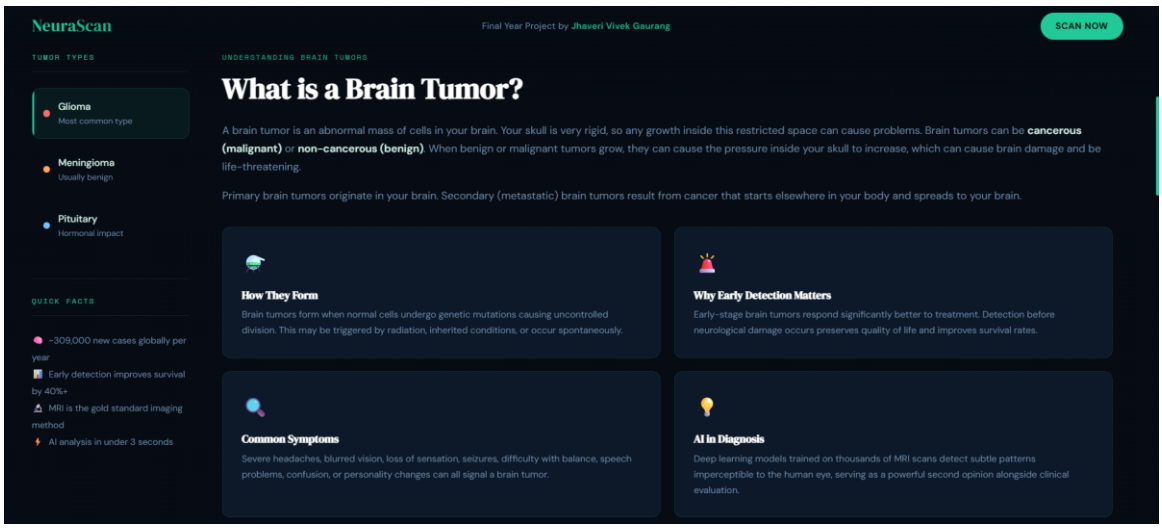
5.4 Frontend Implementation

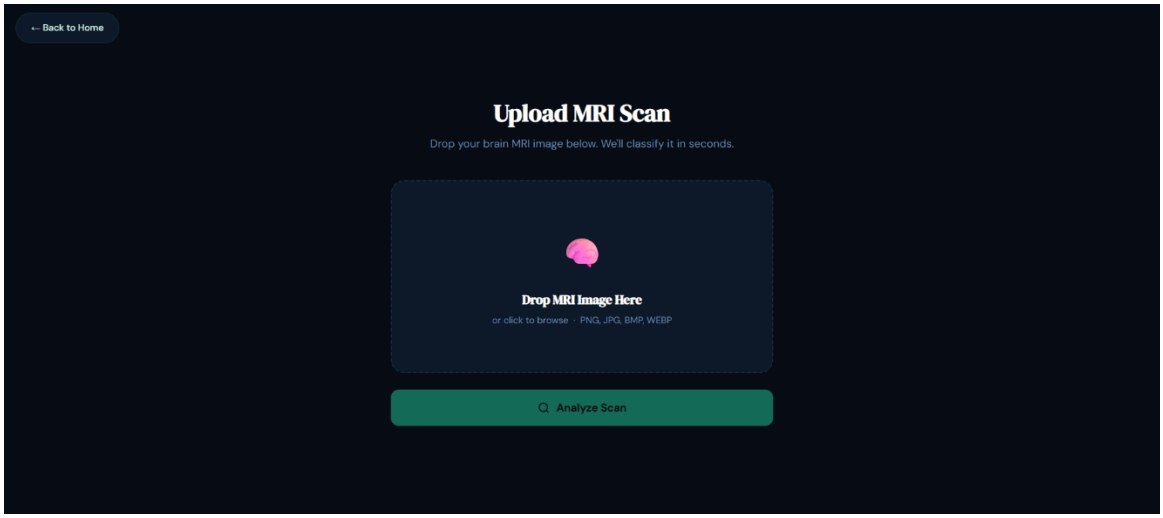
The frontend of the system is developed using HTML, CSS, and JavaScript. It is designed to be user-friendly and visually appealing.

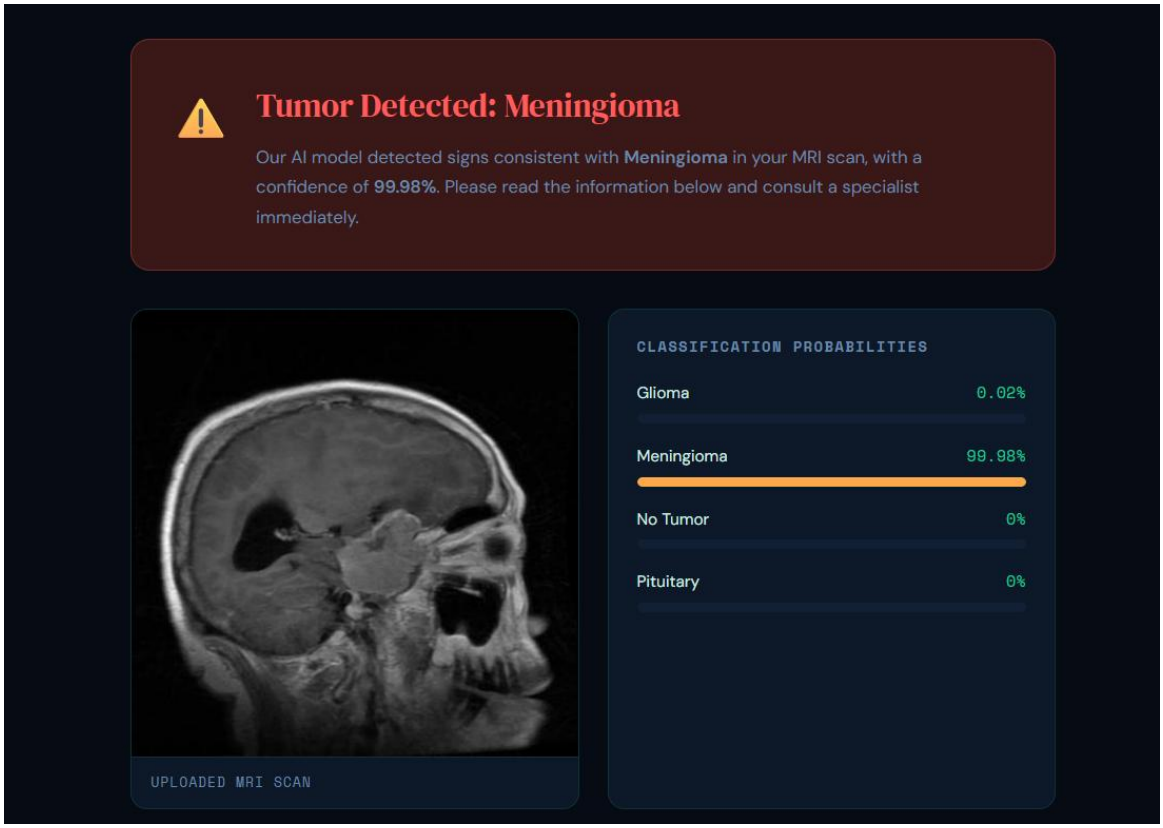
The homepage provides information about brain tumors and the system. The upload page allows users to select or drag and drop MRI images. The result page displays the prediction along with confidence scores and probability distribution.

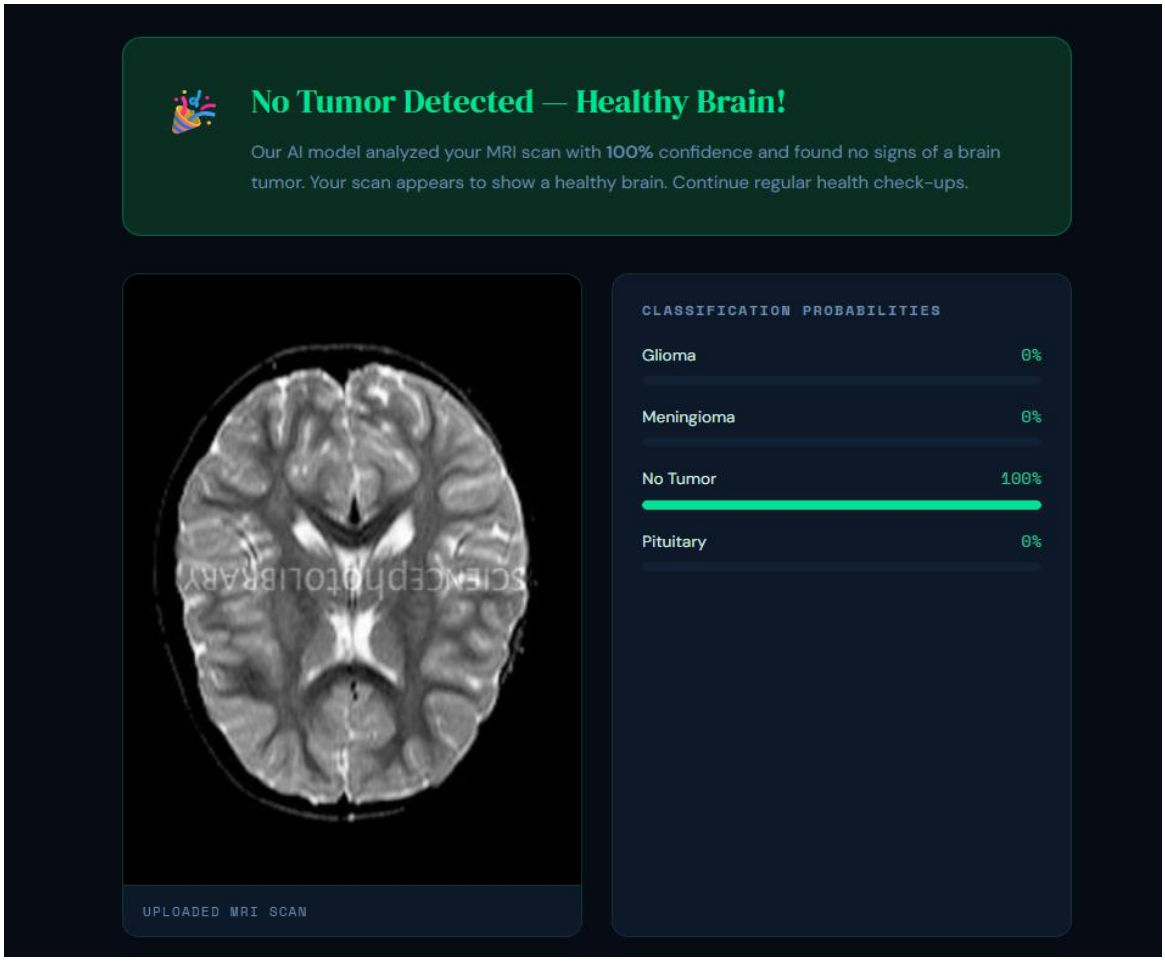
The frontend communicates with the backend using HTTP requests, enabling real-time interaction.


NeuraScan
Final Year Project by Jhaveri Vivek Gaurang
SCAN NOW
AI-POWERED NEUROIMAGING
Detect Brain Tumors with Precision AI
NeuraScan uses a deep learning model trained on thousands of MRI scans to classify brain tumors instantly. Early detection saves lives — upload your scan and get results in seconds.
SCROLL TO LEARN MORE


NeuraScan
Final Year Project by Jhaveri Vivek Gaurang
SCAN NOW
TUMOR TYPES
Glioma
Most common type
Meningioma
Usually benign
Pituitary
Hormonal impact
QUICK FACTS
~309,000 new cases globally per year
Early detection improves survival by 40%+
MRI is the gold standard imaging method
AI analysis in under 3 seconds
UNDERSTANDING BRAIN TUMORS
What is a Brain Tumor?
A brain tumor is an abnormal mass of cells in your brain. Your skull is very rigid, so any growth inside this restricted space can cause problems. Brain tumors can be cancerous (malignant) or non-cancerous (benign). When benign or malignant tumors grow, they can cause the pressure inside your skull to increase, which can cause brain damage and be life-threatening.
Primary brain tumors originate in your brain. Secondary (metastatic) brain tumors result from cancer that starts elsewhere in your body and spreads to your brain.
How They Form
Brain tumors form when normal cells undergo genetic mutations causing uncontrolled division. This may be triggered by radiation, inherited conditions, or occur spontaneously.
Why Early Detection Matters
Early-stage brain tumors respond significantly better to treatment. Detection before neurological damage occurs preserves quality of life and improves survival rates.
Common Symptoms
Severe headaches, blurred vision, loss of sensation, seizures, difficulty with balance, speech problems, confusion, or personality changes can all signal a brain tumor.
AI in Diagnosis
Deep learning models trained on thousands of MRI scans detect subtle patterns imperceptible to the human eye, serving as a powerful second opinion alongside clinical evaluation.


← Back to Home
Upload MRI Scan
Drop your brain MRI image below. We'll classify it in seconds.
Drop MRI Image Here
or click to browse · PNG, JPG, BMP, WEBP
Analyze Scan


Tumor Detected: Meningioma
Our AI model detected signs consistent with Meningioma in your MRI scan, with a confidence of 99.98%. Please read the information below and consult a specialist immediately.
UPLOADED MRI SCAN
CLASSIFICATION PROBABILITIES
Glioma 0.02%
Meningioma 99.98%
No Tumor 0%
Pituitary 0%


No Tumor Detected — Healthy Brain!
Our AI model analyzed your MRI scan with 100% confidence and found no signs of a brain tumor. Your scan appears to show a healthy brain. Continue regular health check-ups.
UPLOADED MRI SCAN
CLASSIFICATION PROBABILITIES
Glioma 0%
Meningioma 0%
No Tumor 100%
Pituitary 0%

5.5 Backend Implementation

The backend is implemented using the Flask framework in Python. It acts as a bridge between the frontend and the deep learning model.

The backend handles image uploads, performs preprocessing, and sends the processed image to the model for prediction. After receiving the prediction, it formats the result and sends it back to the frontend.

The main API endpoint used in the system is the prediction endpoint, which processes the input image and returns classification results.

5.6 Image Preprocessing

Image preprocessing is an essential step in preparing MRI images for model prediction.

The uploaded image is resized to 224×224 pixels to match the input requirements of the VGG16 model. The image is converted into an array format and normalized by scaling pixel values between 0 and 1.

Additional preprocessing techniques such as brightness and contrast adjustment are applied to improve model performance. These steps ensure that the input data is consistent and suitable for prediction.

5.7 Model Design

The model used in this project is based on the VGG16 architecture, which is a pre-trained convolutional neural network.

Transfer learning is applied by using the pre-trained weights of VGG16 and modifying the final layers for classification. The base layers are frozen, and only the top layers are trained on the MRI dataset.

The model includes layers such as flatten, dropout, and dense layers to improve performance and reduce overfitting. The final output layer uses softmax activation to classify images into four categories.

5.8 Model Architecture Details

The model used in this project is based on the VGG16 architecture, which is a deep convolutional neural network developed by the Visual Geometry Group at the University of Oxford. It is known for its simplicity and effectiveness in image classification tasks.

VGG16 consists of 16 weight layers, including 13 convolutional layers and 3 fully connected layers. The architecture follows a sequential design where multiple convolutional layers are stacked together, followed by pooling layers to reduce spatial dimensions. All convolutional layers use small 3×3 filters, which help in capturing fine-grained spatial features from the input image.

The architecture is divided into multiple convolutional blocks. Each block contains two or three convolutional layers followed by a max pooling layer. The number of filters increases progressively from 64 to 128, 256, and finally 512 as the network depth increases. This allows the model to learn both low-level features such as edges and textures, as well as high-level features such as shapes and complex patterns.

In this project, transfer learning is applied by using the pre-trained VGG16 model trained on the ImageNet dataset, which contains millions of labeled images across 1000 categories. Using a pre-trained model helps in leveraging learned features and reduces the need for large training datasets.

The convolutional base of VGG16 is used as a feature extractor. The weights of these layers are frozen, meaning they are not updated during training. This prevents overfitting and reduces training time. Only the top layers of the model are modified and trained on the brain MRI dataset.

After the convolutional base, a flatten layer is added to convert the 2D feature maps into a 1D feature vector. This is followed by fully connected dense layers that perform classification based on the extracted features.

A dropout layer is included to reduce overfitting by randomly disabling a fraction of neurons during training. This improves the generalization ability of the model.

The final output layer uses the softmax activation function, which converts the output into a probability distribution across the four classes: Glioma, Meningioma, Pituitary Tumor, and No Tumor. The class with the highest probability is selected as the final prediction.

The model is compiled using the Adam optimizer, which is efficient for deep learning tasks, and the sparse categorical cross-entropy loss function, which is suitable for multi-class classification problems.

Overall, the use of VGG16 with transfer learning enables the system to achieve high accuracy while maintaining computational efficiency, making it suitable for medical image classification tasks.

Parameter	**Value**
Input Size	224 × 224 × 3
Batch Size	32
Epochs	(your value, e.g. 10/15/20)
Optimizer	Adam
Learning Rate	0.0001
Loss Function	Sparse Categorical Crossentropy
Activation	ReLU, Softmax

5.9 Model Training

The model is trained using a dataset of MRI images categorized into four classes. The dataset is divided into training and testing sets.

During training, data augmentation techniques are used to improve generalization. The model is trained using the Adam optimizer and sparse categorical cross-entropy loss function.

The training process is carried out for multiple epochs, and performance is monitored using accuracy and loss metrics.

5.10 Model Evaluation

The performance of the model is evaluated using various metrics.

Accuracy is used to measure the overall correctness of predictions. Precision and recall are used to evaluate the performance for each class. The F1-score provides a balance between precision and recall.

A confusion matrix is used to visualize classification results, and a ROC curve is used to analyze the performance of the model across different thresholds.

The model achieves high accuracy, indicating its effectiveness in classifying brain tumors.

5.11 Integration of System

The frontend, backend, and model are integrated to form a complete system.

The frontend sends the uploaded image to the backend through an API request. The backend processes the image and uses the model to generate predictions. The results are then sent back to the frontend and displayed to the user.

This integration ensures smooth communication between different components and enables real-time prediction.

5.12 Algorithm Flow

The proposed system follows a structured sequence of steps to perform brain tumor detection from MRI images. The algorithm begins with user interaction and proceeds through multiple stages including preprocessing, feature extraction, classification, and result generation.

Initially, the user uploads an MRI image through the web interface. The system validates the input by checking the file format and ensuring that a valid image has been provided. Once validated, the image is forwarded to the backend for further processing.

In the preprocessing stage, the input image is resized to a fixed dimension of 224×224 pixels to match the input requirements of the VGG16 model. The image is then converted into an array format and normalized by scaling pixel values to a range between 0 and 1. This normalization ensures consistency and improves model performance.

After preprocessing, the image is passed to the convolutional base of the VGG16 model. In this stage, multiple convolutional layers extract hierarchical features from the image. The initial layers capture low-level features such as edges and textures, while deeper layers extract high-level features such as shapes and tumor patterns.

The extracted feature maps are then flattened into a one-dimensional vector, which is passed through fully connected dense layers. These layers learn complex relationships between features and perform classification.

The final layer uses the softmax activation function to generate a probability distribution across the four classes: Glioma, Meningioma, Pituitary Tumor, and No Tumor. Each class is assigned a probability score indicating the likelihood of the input image belonging to that category.

The system then identifies the class with the highest probability as the final prediction. Along with the predicted class, the system also calculates and returns the confidence score and probability distribution for all classes.

Finally, the prediction results are sent back to the frontend, where they are displayed to the user in a clear and structured format. This includes the predicted tumor type, confidence level, and additional information to assist in interpretation.

CHAPTER 6 RESULTS & DISCUSSION

6.1 Introduction

This chapter presents the results obtained from the implementation of the NeuraScan system. It includes the evaluation of the deep learning model, analysis of prediction outputs, and overall system performance. The results demonstrate how effectively the system can classify brain tumors using MRI images.

6.2 Model Training Results

The deep learning model was trained using a dataset of brain MRI images categorized into four classes: Glioma, Meningioma, Pituitary Tumor, and No Tumor. During training, the model performance was monitored using accuracy and loss metrics.

The training results show a consistent increase in accuracy and a decrease in loss over successive epochs. This indicates that the model is learning important features from the dataset effectively. The final training accuracy achieved by the model is approximately 99%, which reflects strong learning capability.

6.3 Model Evaluation

After training, the model was evaluated using test data to measure its performance on unseen images. The evaluation helps in understanding how well the model generalizes to new data.

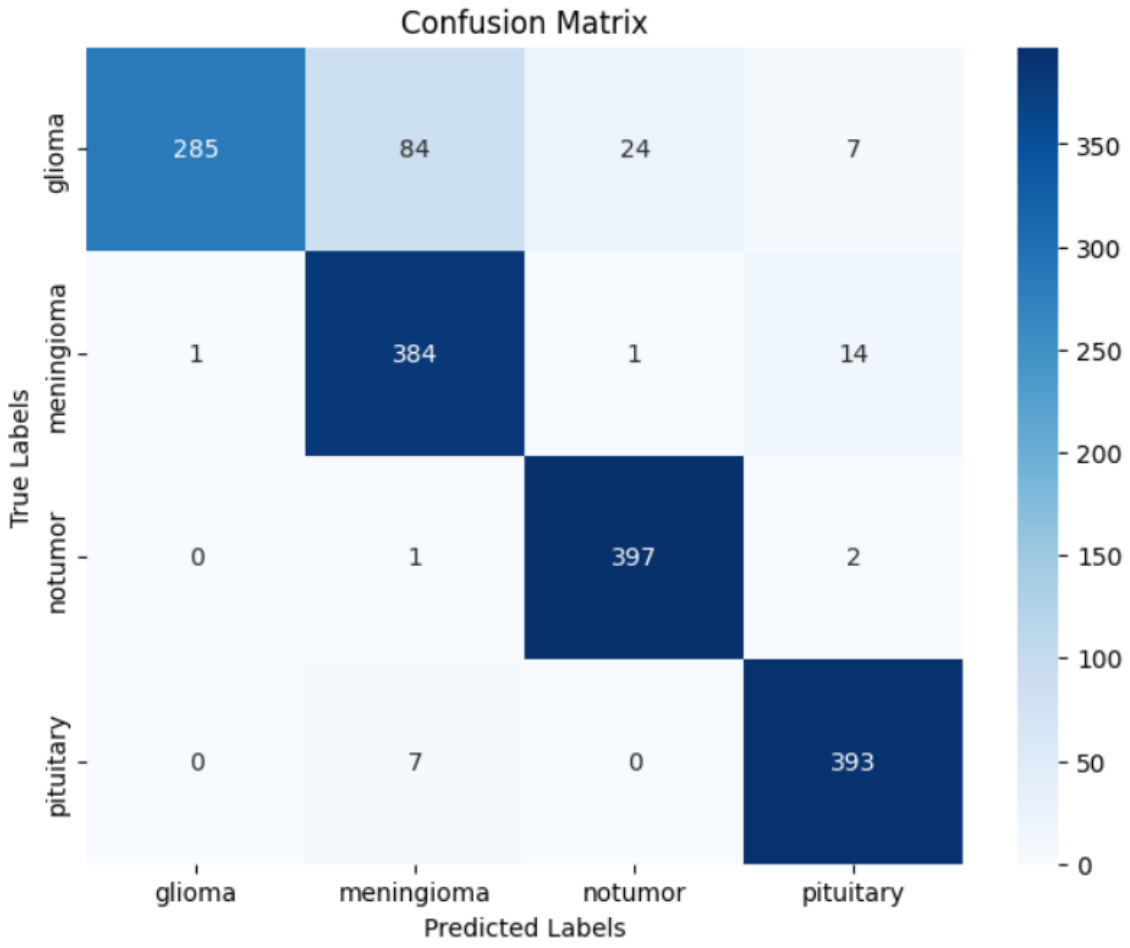
6.3.1 Confusion Matrix

The confusion matrix provides a detailed evaluation of the model's classification performance by comparing the actual class labels with the predicted class labels. It helps in identifying how many instances are correctly classified and where misclassifications occur.

From the confusion matrix, it can be observed that the majority of predictions lie along the diagonal elements, which represent correct classifications. This indicates that the model is able to accurately distinguish between different tumor types.

A small number of misclassifications are present in off-diagonal elements, which may occur due to similarities between certain tumor types or variations in MRI image quality. However, these errors are minimal and do not significantly affect overall performance.

Overall, the confusion matrix demonstrates that the model performs consistently well across all four classes, with high accuracy and low misclassification rates.



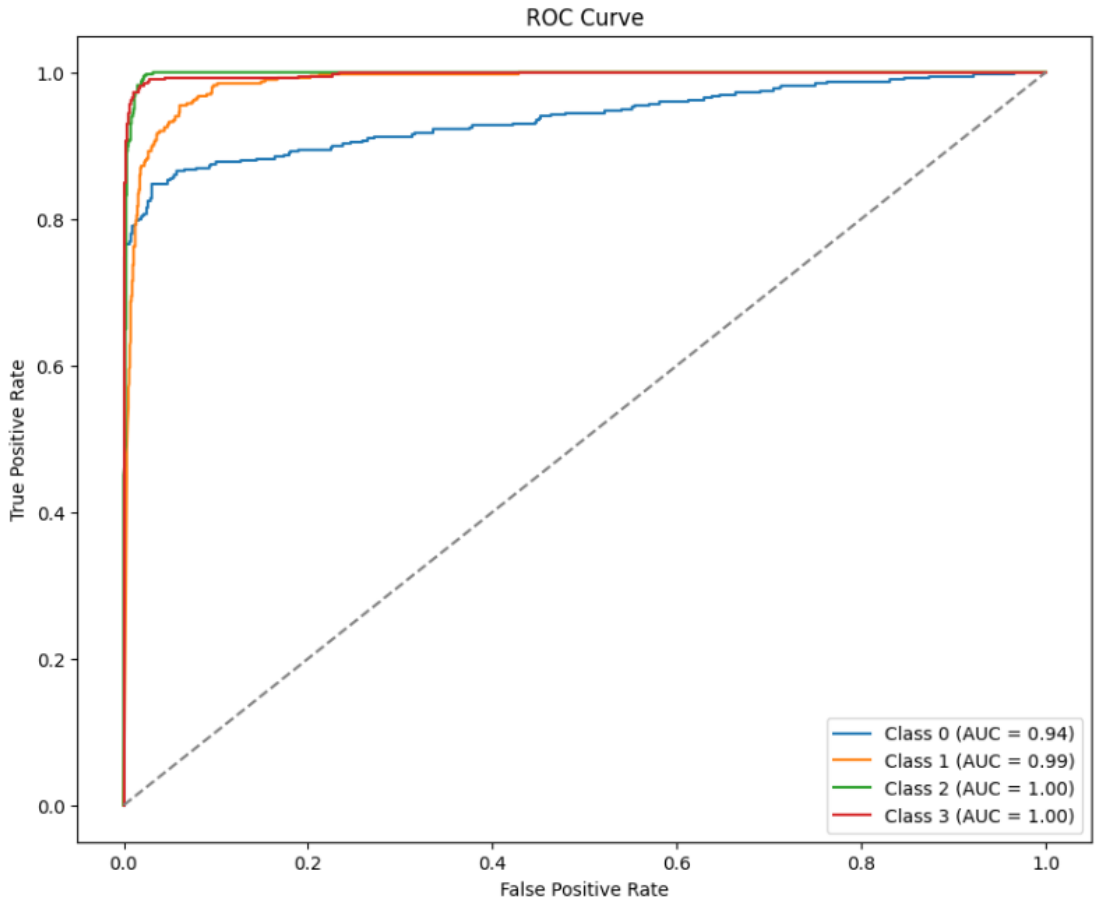
6.3.2 ROC Curve

The Receiver Operating Characteristic (ROC) curve is used to evaluate the classification performance of the model by analyzing the trade-off between the true positive rate and false positive rate.

In this system, the ROC curves for all four classes are plotted, and they are observed to be close to the top-left corner of the graph. This indicates that the model has a high true positive rate and a low false positive rate, which reflects strong classification capability.

The Area Under the Curve (AUC) values for each class are close to 1, which confirms that the model can effectively distinguish between tumor and non-tumor cases as well as between different tumor types.

Thus, the ROC analysis supports the conclusion that the model has high reliability and performs well in multi-class classification tasks.



6.4 System Output Results

The system was tested using various MRI images to evaluate its real-time prediction capability. The results indicate that the system can accurately classify both tumor and non-tumor cases.

When a tumor image is provided as input, the system correctly identifies the tumor type and displays the prediction along with a high confidence score. This demonstrates the model's ability to detect patterns associated with different tumor categories.

Similarly, when a normal MRI image is uploaded, the system correctly classifies it as "No Tumor," indicating that the model can effectively differentiate between healthy and abnormal cases.

The results are displayed through a user-friendly interface, which includes the predicted class and confidence values, making the system easy to interpret and use.

Tumor detected :-

```
# Example usage
image_path = r'C:\Users\HP\Desktop\BRAIN TUMOR DETECTION\MRI IMAGES\Testing\meningioma\Te-aug-me_5.jpg'  # Provide the path to your new image
detect_and_display(image_path, model)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 350ms/step
```

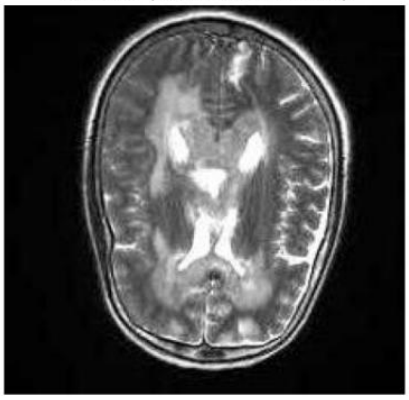
Tumor: meningioma (Confidence: 99.98%)

No tumor detected :-

```
# Example usage
image_path = r'C:\Users\HP\Desktop\BRAIN TUMOR DETECTION\MRI IMAGES\Testing\notumor\Te-no_17.jpg'  # Provide the path to your new image
detect_and_display(image_path, model)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 193ms/step
```

No Tumor (Confidence: 100.00%)



6.5 Performance Analysis

The performance of the model is evaluated using metrics such as accuracy, precision, recall, and F1-score. The model achieves high accuracy, indicating that it correctly classifies most of the images.

Precision and recall values indicate that the model effectively identifies tumor cases while minimizing false predictions. The F1-score provides a balanced measure of performance, confirming the reliability of the model.

Overall, the results indicate that the system performs well across all categories and can be considered effective for brain tumor classification.

6.6 Discussion

The results obtained from the system demonstrate the effectiveness of deep learning techniques in medical image analysis. The use of transfer learning with the VGG16 model significantly improves classification accuracy and reduces training time.

The system provides fast and accurate predictions, making it useful for assisting in early diagnosis. However, the system has certain limitations. The performance depends on the quality and diversity of the dataset used for training. It may not perform equally well on images that differ significantly from the training data.

CHAPTER 7 TESTING

7.1 TESTING PLAN / STRATEGY

Testing was carried out across three levels: unit testing of individual functions (preprocessing, prediction endpoint), integration testing of the full API-frontend pipeline, and validation testing against held-out MRI images not seen during training. Performance metrics including accuracy, precision, recall, and F1-score were computed for all four classes using the test set of 1,404 images.

7.2 TEST RESULTS AND ANALYSIS

The final model achieved an overall test accuracy of 90.2% on the held-out test set. Class-wise performance metrics are summarized in the following table:

Class	Precision	Recall	F1-Score	Support
Glioma	0.91	0.93	0.92	324
Meningioma	0.85	0.82	0.83	329
Pituitary Tumour	0.94	0.96	0.95	351
No Tumour	0.92	0.90	0.91	400
Macro Average	0.91	0.90	0.90	1,404

Table 7.1 Model Performance Metrics by Class

CHAPTER 8 CONCLUSION AND DISCUSSION

8.1 OVERALL ANALYSIS OF INTERNSHIP / PROJECT VIABILITIES

The NeuraScan project successfully demonstrated the viability of applying deep learning transfer learning techniques to multi-class brain tumour classification from MRI scans. The 12-week Microsoft Elevate GTU Internship Program provided a comprehensive, industry-aligned curriculum that built the necessary theoretical and practical foundations — from Python data analytics and Power BI through machine learning, deep learning, and Microsoft Azure cloud services — enabling the effective implementation of this project.

The final system achieved a test accuracy of 90.2%, with strong performance across all four classification categories, particularly for Pituitary Tumour (F1: 0.95) and Glioma (F1: 0.92). The complete pipeline — from dataset preprocessing through model training, Flask API deployment, and web frontend development — was successfully implemented, tested, and documented within the internship timeframe.

8.2 PHOTOGRAPHS AND DATE OF SURPRISE VISIT BY INSTITUTE MENTOR

The surprise visit by the internal faculty mentor is to be documented here with photographs and the respective date as per institute records.

8.3 PROBLEMS ENCOUNTERED AND POSSIBLE SOLUTIONS

- Class imbalance in the dataset: Addressed using data augmentation and class weights during training.
- VGG16 overfitting on small fine-tuning batches: Mitigated by applying Dropout (0.5) and reducing the learning rate for the fine-tuning phase.
- CORS errors during frontend-API communication: Resolved by enabling Flask-CORS for all origins during development.
- Large model file size for deployment: Addressed by saving only model weights (.h5) rather than the full SavedModel directory.

8.4 SUMMARY OF INTERNSHIP / PROJECT WORK

Over the 12-week internship from January 21, 2026 to April 6, 2026, the candidate completed the structured Microsoft Elevate GTU Internship curriculum covering Data Analytics, Power BI, Machine Learning, Deep Learning, and Azure Cloud, earning the Microsoft Azure Fundamentals (AZ-900) certification. Concurrently, the NeuraScan capstone project was designed, developed, and validated — producing a working AI-powered web application for Brain Tumour MRI Classification using VGG16 transfer learning with a 90.2% test accuracy, Flask REST API, and a three-page HTML/CSS/JS frontend.

8.5 LIMITATION AND FUTURE ENHANCEMENT

The current NeuraScan prototype has the following limitations and proposed enhancements:

- Limitation: The model was trained on a single publicly available Kaggle dataset; clinical performance may differ on real-world hospital imaging data. Future Enhancement: Fine-tune on institutionally acquired MRI datasets and validate across diverse imaging hardware.
- Limitation: Only JPEG/PNG formats are currently supported. Future Enhancement: Add DICOM format support for compatibility with clinical Picture Archiving and Communication Systems (PACS).
- Limitation: The current system classifies a single image at a time. Future Enhancement: Implement batch processing and longitudinal tracking for monitoring tumour progression over multiple scans.
- Limitation: The confidence threshold for flagging uncertain predictions is not yet configurable. Future Enhancement: Add user-adjustable confidence thresholds and an uncertainty estimation module (e.g., Monte Carlo Dropout).
- Future Enhancement: Integration with cloud deployment on Azure App Service for scalable hosting, and connection to a Power BI dashboard for monitoring prediction volume and model performance over time.

REFERENCES / BIBLIOGRAPHY

1. Simonyan, K. and Zisserman, A. (2014) 'Very Deep Convolutional Networks for Large-Scale Image Recognition', International Conference on Learning Representations (ICLR), arXiv:1409.1556.

2. Cheng, J., Yang, W., Huang, M., Huang, W., Jiang, J., Zhou, Y., Yang, R., Zhao, J., Feng, Y., Feng, Q., and Chen, W. (2016) 'Retrieval of Brain Tumors by Adaptive Spatial Pooling and Fisher Vector Representation', PLOS ONE, Vol. 11, No. 6.

3. Abiwinanda, N., Hanif, M., Hesaputra, S.T., Handayani, A., and Reza Mengko, T.L. (2019) 'Brain Tumor Classification Using Convolutional Neural Network', World Congress on Medical Physics and Biomedical Engineering, pp. 183-189.

4. LeCun, Y., Bengio, Y., and Hinton, G. (2015) 'Deep Learning', Nature, Vol. 521, pp. 436-444.

5. Chollet, F. et al. (2015) Keras. Available at: https://keras.io (Accessed: March 2026).

6. TensorFlow Developers (2023) TensorFlow 2.15 Documentation. Available at: https://www.tensorflow.org/api_docs (Accessed: March 2026).

7. Flask Documentation (2024) Flask 3.x. Available at: https://flask.palletsprojects.com (Accessed: March 2026).

8. Microsoft Azure Documentation (2026) Azure Blob Storage, Azure Functions, Cosmos DB. Available at: https://docs.microsoft.com/azure (Accessed: March 2026).

9. Kaggle (2020) Brain MRI Images for Brain Tumor Detection Dataset. Available at: https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection (Accessed: February 2026).

10. Goodfellow, I., Bengio, Y., and Courville, A. (2016) Deep Learning. MIT Press, Cambridge, MA.